Project 2 on Neural networks

Tim Herrmann Holmen

FYS-STK4155
Department of Physics

This project ...

CONTENTS

I. Introduction 2
A. Motivation 2
B. What i have done 2
1. Regression problem 2
2. Classification problem 2
C. In this report 2

II. Theory 2
A. Cost functions and their gradients 2
1. MSE 2
2. Binary cross entropy: log loss 2
3. Multiclass cross entropy cost function 3
B. Activation functions and their gradients 3
1. Sigmoid logit function 3
2. RELU function 3

III. Methods 3
A. NeuralNetwork 3
1. Feed-forward pass 4
2. Backpropagation 4
B. Stochastic Gradient Descent 4
C. Comparison with OLS 4
D. Activation, depth and width 4
1. Ridge 4
2. Lasso 4
E. Classification 4
F. Measure of quality 4
1. Confusion matrix 4
G. ROC and AUC 4

IV. Results 4

V. Discussion 4

VI. Conclusion 4

A. Use of AI/LLM Tools 4
1. Tools used 4
2. Text contributions 4
3. Code contributions 5
a. LLM Declaration this section: 5

References 5

I. INTRODUCTION

A. Motivation

Aritficial neural networks have shown great results and improvements in the last years, impacting fields such as science, poltics and philosophy. Large language models, image detection, drones and autonomous systems, all have variations of artificial neural networks as its basis. In politics we can hear about intelligent machines, aritificial intelligence and super intelligence. The motivation for this project is to learn about neural networks, demistify its supernatural appearance, and understand the mechanics and its capacity to learn and solve known functions and data relations. The method and results will be compared to known methods of linear regression and classification. As a test of strength and calibration.

B. What i have done

1. Regression problem

We have created a feed forward neural network; FFNN, used stochastic gradient descent; SGD, to find the minimum of the cost function, which is the MSE for the regression problem. The gradients are calculated using backpropagation, which uses the kernal rule of derivation on the FFNN in reverse; from the output and target MSE derivative to the derivative of the parameters of the first layer. The gradients are compared with jax, automatic differentiation, to ensure agreement. In the SGD the descent method used on the parameters and gradients is the Plain gradient descent; regular updating, RMSprop; complex updating, Adam; best updating. Different layer width and depths are explored, along with the activation functions in the hidden layers, different learning rates are explored and compared. The output layer has the identity as activaiton function. The choice of activation function is the identity since the problem of choice is a regression problem. This network is compared with OLS for the one dimensional runge function. Further L1 and L2 penalties are applied to the cost function, with a variable penalty value λ, and these models are compared with Lasso and Ridge.

2. Classification problem

For the classification problem the difference is the output activation function and the cost function. The output function is the softmax function, which.. The cost function is the cross entropy, and the derivative of this function changes the backpropagation algorithm accordingly. This and the data set and following problem is the difference between this network and the regression network. The study problem is the multiclass MNIST problem of handwritten digits (LeCun, Cortes and Burges), the description of the data set is found at https://www.tensorflow.org/datasets/catalog/mnist. For the classification network the explored parameters are whidth of the hidden layer, the learning rate γ, the gradient methods plain, rmsprop and adam. Also the L1 and L2 penalties on the cost function are applied with different penalty values λ.

C. In this report

II III IV V VI

II. THEORY

A. Cost functions and their gradients

The loss function is the most important function for training, since training a model means to minimize the cost function of the model. To minimize the cost function, we use backpropagation to find the gradient of the cost, with regards to and gradient descent.

1. MSE

When we have our calculated output value $\tilde{y}$ and the true output value y, with n data points, the mean-squared error and its derivative is

$$MSE(y,\tilde{y}) = \frac{1}{n}\sum_{i=0}^{n-1}(y-\tilde{y})^2 \quad \frac{1}{n}||y - X\Theta||_2^2 \tag{1}$$

a. L2 norm which corresponds to the Ridge cost function.

$$MSE + \lambda||\Theta||_2^2 \quad \Theta(W,b) \tag{2}$$

b. L1 norm which corresponds to the Lasso cost function.

$$MSE + \lambda||\Theta|| \quad \Theta(W,b) \tag{3}$$

(Hjorth-Jensen, 2016, p. 86-91)

2. Binary cross entropy: log loss

With two classes, we assume a binary output $y \in \{0,1\}$, and a predicted probability $p \in [0,1]$ which is the output of the sigmoid function of z_i. Where $z_i = x_i^T\Theta$ is the linear combination of the input features and the

model parameters Θ. For a single example, data point i, the binary cross entropy cost function is defined as

$$C(\Theta) = -\sum_{i=1}^{n}(y_i \log p_i + (1-y_i)\log[1-p_i]) \quad (4)$$

$$p_i = \sigma(z_i) = \frac{1}{1+\exp(-z_i)} \quad (5)$$

$$(6)$$

The cost function is derived from the maximum likelihood estimation of the Bernoulli distribution, the negative log likelihood of the Bernoulli distribution. This is known as the cross entropy, and it is the loss function used to train binary classification models. With the linear function $z_i = x_i^T\Theta$, the gradient of the cost function with respect to the model parameters Θ is given by

$$\frac{dC(\theta)}{d\theta} = -X^T(y-p) \quad (7)$$

For N data points, the mean binary cross entropy is defines as

$$\frac{1}{N}\sum_{i=1}^{N} C(\theta)_i$$

And the term $1/N$ follows through te calculations to give the mean binary cross entropy gradient, which is given by

$$\nabla_\Theta C(\Theta) = -\frac{1}{n}X^T(y-p) \quad (8)$$

The cross entropy L1 and L2 norm is supplemented like the ordinary MSE.

3. Multiclass cross entropy cost function

For k klasses, one uses the soft max function to calculate the probability of each class. Again we have the linear function $z_k = X^T\Theta_k$ for each class, over all features and data points, the softmax functions is defined as

$$p(C=k \mid x) = \frac{\exp(z_k)}{\sum_{l=0}^{K-1}\exp(z_l)} k = 0, \ldots, K-1 \quad (9)$$

The nominator is for class k, and the denominator is the sum of all classes. The multiclass cross entropy cost function is defined as

$$C(\Theta) = -\sum_{i=1}^{n}\sum_{k=0}^{K-1} y_{ik}\log p(C=k|x_i) \quad (10)$$

Which is summed over all data points and classes. The gradient of the multiclass cross entropy cost function is given by

$$\frac{dC}{d\Theta_k} = -X^T(y_k - p_k) \quad (11)$$

The cross entropy L1 and L2 norm is supplemented like the ordinary MSE. Stability: The softmax function can be numerically unstable for large values of z_k, for a large value the e exponential function can overflow. Since the softmax is invariant udner subtracting a constant frome every score, one subtracts the largest before exponentiating:

$$p_k = \frac{\exp(z_k - z_{max})}{\sum_l \exp(z_l - z_{max})}$$

, $z_k = x^T\Theta_k$. (Hjorth-Jensen, 2016, p. 151-156)

B. Activation functions and their gradients

1. Sigmoid logit function

$$\sigma(z) = \frac{1}{1+\exp(-z)} \quad (12)$$

$$\sigma'(z) = \sigma(z)(1-\sigma(z)) \quad (13)$$

2. RELU function

$$ReLU(z) = \max(0, z) \quad (14)$$

$$ReLU'(z) = \begin{cases} 0 & z < 0 \\ 1 & z > 0 \end{cases} \quad (15)$$

(Hjorth-Jensen, 2016, p. 214-242)

III. METHODS

A. NeuralNetwork

The Neural network creates the architecture, predicts by feed forward, and calculates the cost function based on the input values and the target values. It is a class which is initiated with input size, output sizes, activation functions and its derivatives, the cost function and its derivative, and the seed which ensures consistency when creating the random initial values (rng) of the weights. The chosen seed is 2026 for the regression problem. The architecture of the layers is created inside the class and saved as an attribute, by iterating over the output sizes and creating the initial weights and biases. for output size in output sizes

$$W = standard_normal(inputsize, outputsize)$$

$$b = zeros(outputsize)$$

input size = output size layers append ((W, b))

1. Feed-forward pass

This first part of the network is the feed forward, it sends he input values through the layers, each layers activation function is applied and the output layer is calculated by the chosen activaiton function, which is the identity for the regression problem. The feed-forward procedure is as follows:

$$\mathbf{a}^{(0)} = \mathbf{x}, \tag{16}$$

$$\mathbf{z}^{(l)} = \mathbf{a}^{(l-1)} W^{(l)} + \mathbf{b}^{(l)}, \tag{17}$$

$$\mathbf{a}^{(l)} = \phi^{(l)}\Big(\mathbf{z}^{(l)}\Big), \qquad l = 1, \ldots, L, \tag{18}$$

$$\hat{\mathbf{y}} = \mathbf{a}^{(L)}. \tag{19}$$

For backpropagation, it is important to save the input values, the z values and the output values a during the feed forward.

2. Backpropagation

The backpropagation is the most important part of the neural network, as it is this process that creates the gradients of each layer, which are used to update the weights and biases. This process is different depending on the cost function, since the derivative of the cost function is the value we want to minimize. Thus it is different for the regression problem, which uses mean squared error, and the classification which uses cross entropy. The input layers, the z values and the output values a is generated by the feed forward pass saver, the backpropagation iterates backwards, from the last layer to the first. Regressor, mean squared error

$$\text{Output layer:} \quad \frac{\partial C}{\partial a^{(L)}} = \text{costfunction derivative}, \tag{20}$$

$$\text{Hidden layer:} \quad \frac{\partial C}{\partial a^{(l)}} = \delta^{(l+1)} \left(W^{(l+1)}\right)^{\top}, \tag{21}$$

$$\delta^{(l)} = \frac{\partial C}{\partial z^{(l)}} = \frac{\partial C}{\partial a^{(l)}} \odot \sigma'\Big(z^{(l)}\Big), \tag{22}$$

$$\frac{\partial C}{\partial W^{(l)}} = \left(a^{(l-1)}\right)^{\top} \delta^{(l)}, \tag{23}$$

$$\frac{\partial C}{\partial b^{(l)}} = \sum_{i=1}^{N} \delta_i^{(l)}, \tag{24}$$

$$\nabla^{(l)} = \left(\frac{\partial C}{\partial W^{(l)}}, \frac{\partial C}{\partial b^{(l)}}\right). \tag{25}$$

Cross entropy, classification.

$$\text{Output layer:} \quad \Delta^{(L)} = \frac{\partial C}{\partial Z^{(L)}} = \frac{A^{(L)} - Y}{n}, \tag{26}$$

$$\text{Hidden layers:} \Delta^{(l)} = \frac{\partial C}{\partial Z^{(l)}} \tag{27}$$

$$= \Big(\Delta^{(l+1)} (W^{(l+1)})^{\top}\Big) \tag{28}$$

$$\odot \sigma'\Big(Z^{(l)}\Big), \tag{29}$$

$$l = L-1, \ldots, 1. \tag{30}$$

B. Stochastic Gradient Descent

C. Comparison with OLS

D. Activation, depth and width

1. Ridge

2. Lasso

E. Classification

F. Measure of quality

1. Confusion matrix

G. ROC and AUC

IV. RESULTS

V. DISCUSSION

VI. CONCLUSION

Appendix A: Use of AI/LLM Tools

1. Tools used

List every LLM tool used for the project, including its model or version, how it was accessed, and the approximate dates of use.

- *[Tool/version; access method; date used]*
- *GPT 6-Luna (codex), used 22.10–06.11.2026*
- *[GPT 6-Luna/version; GitHub Copilot (coding assistant session);* 22.10 − 06.11.2026*]*

2. Text contributions

For each report section, enter the highest applicable contribution level and briefly describe the assistance. Use level 0 for no assistance, level 1 for language-only edits, level 2 for editorial changes, and level 3 for substantial drafting. Replace the placeholders with an accurate record.

Abstract:: Level *[0–3]*; *[Describe assistance, or write "none"]*

Introduction:: Level *[0–3]*; *[Describe assistance, or write "none"]*

Theory:: Level *[0–3]*; *[Describe assistance, or write "none"]*

Methods:: Level *[0–3]*; *[Describe assistance, or write "none"]*

Results:: Level *[0–3]*; *[Describe assistance, or write "none"]*

Discussion:: Level *[0–3]*; *[Describe assistance, or write "none"]*

Conclusion:: Level *[0–3]*; *[Describe assistance, or write "none"]*

Level 3 prompt and revision note (delete if not applicable): [Summarise the prompt or task, how the generated text was changed, and how it was checked.]

3. Code contributions

List each code file or notebook included in the submission. Use level 0 for no assistance, level 1 for debugging, level 2 for a snippet, level 3 for a skeleton, and level 4 for substantial generation. Include what was tested, adapted, or verified.

`[Path / filename]::` Level *[0–4]*; *[Describe assistance and your testing/adaptations]*

Duplicate the code entry above for every submitted code file or notebook.
For every LLM-assisted function at level 2 or above, add an `LLM-assisted` note to its docstring. Add an inline note for assisted code blocks shorter than a function, and a Markdown note above assisted notebook cells.

a. LLM Declaration this section:

The skeleton of this section was created using GitHub Copilot. (modelversion unknown; coding-assistant session, November 6, 2026).

REFERENCES

M. Hjorth-Jensen (2016), *Machine Learning and the Physical Sciences* (Springer).